SQL Set Operations Comprehensive Guide

A Practical Handbook for Vertical Data Selection & Multi-Query Logic

Set operations combine the results of two or more `SELECT` statements into one single result. Instead of joining tables horizontally (adding columns based on a matching key), set operations combine results **vertically** (adding rows). Each `SELECT` statement produces a set of rows, and the set operation defines what to do with those sets—whether to keep everything, keep only common values, or extract differences.

The Golden Rules — Applying to All Set Operations

Before writing any set operation query, these architectural constraints must be satisfied, or the database engine will throw a syntax/structural error:

> **Rule 1 — Identical Degree (Column Count):** Both `SELECT` statements must return the exact same number of columns. If the first statement contains 3 columns, the second must also map precisely to 3 columns.

> **Rule 2 — Compatible Data Types:** Columns in matching positions must possess compatible data types. Column 1 of the first `SELECT` must align with Column 1 of the second `SELECT`. You cannot combine a `NUMBER` column with a `DATE` column in the same position.

> **Rule 3 — Dominance of First Query Schema:** The column names (or structural aliases) in the final result dataset are derived exclusively from the first `SELECT` statement. Metadata names in subsequent queries are ignored.

> **Rule 4 — Terminal Sorting:** You can only specify a single `ORDER BY` clause, and it must be appended after the final `SELECT` block. Sorting is strictly prohibited inside individual queries forming the set chain.

1. UNION — Combine & Deduplicate

`UNION` combines all rows from both queries and automatically removes duplicate records from the final dataset. If identical records appear across multiple source tables, they are condensed into a single unique instance.

Use Case:

When you require a consolidated list from independent sources where redundant occurrences obscure clarity (e.g., extracting an exhaustive listing of distinct regions representing either employee residences or physical corporate departments).

```
-- Syntax Blueprint
SELECT column1, column2 FROM table_a
UNION
SELECT column1, column2 FROM table_b;
```

Practical HR Schema Example:

Combining active operational job designations alongside Master Catalog definitions to isolate unique titles:

```
SELECT job_id    AS "Job ID",
       'Current' AS "Source"
FROM   employees
UNION
SELECT job_id,
       'Master List'
FROM   jobs
ORDER BY "Job ID";
```

2. UNION ALL — Comprehensive Aggregation

`UNION ALL` functions analogously to `UNION`, except it preserves every single record matching the query criteria without assessing uniqueness. Duplicated items remain explicitly repeated in the final view.

Use Case:

When tracking total activity frequency, preserving exact histories, or optimizing resource overhead. Because `UNION` performs an explicit sorting and scanning sequence to drop duplicates, `UNION ALL` skips this step entirely and achieves significantly higher execution speeds.

```
-- Syntax Blueprint
SELECT column1, column2 FROM table_a
UNION ALL
SELECT column1, column2 FROM table_b;
```

Practical Corporate Matrix Example:

```
SELECT fname || ' ' || lname  AS "Name",
       'Employee'             AS "Type",
       dno                    AS "Dept No"
FROM   employee
UNION ALL
SELECT dependent_name,
       'Dependent',
       NULL
FROM   dependent
ORDER BY "Type", "Name";
```

The Critical Performance Threshold: UNION vs. UNION ALL

If an enterprise database context contains 5 employees with `job_id = 'IT_PROG'` and the jobs master table contains 1 configuration entry:

- **UNION:** Yields exactly 1 row for `'IT_PROG'` due to internal sorting deduplication.
- **UNION ALL:** Yields 6 rows (5 from active staff + 1 configuration row). Always default to `UNION ALL` if data sets are structurally distinct to maximize query velocity.

3. INTERSECT — Overlapping Datasets Only

`INTERSECT` drops records exclusive to individual tables, returning solely the matching records existing concurrently within **both** execution contexts.

Use Case:

Isolating elements that bridge dual business conditions—such as tracking users who maintain active customer profiles while simultaneously holding staff privileges.

```
-- Syntax Blueprint
SELECT column1, column2 FROM table_a
INTERSECT
SELECT column1, column2 FROM table_b;
```

Practical HR Schema Example (Identifying Active Staff Managers):

```
SELECT employee_id                   AS "ID",
       first_name || ' ' || last_name AS "Name"
FROM   employees
INTERSECT
SELECT manager_id,
       first_name || ' ' || last_name
FROM   departments
JOIN   employees ON departments.manager_id = employees.employee_id
ORDER BY "ID";
```

4. EXCEPT / MINUS — Relative Exclusion

Returns records present in the first query that cannot be found anywhere within the parameters of the secondary query. Records unique to the second block are fully dropped.

Dialect Notation: Standard SQL definitions specify **`EXCEPT`** (supported by PostgreSQL, SQL Server). Oracle databases use the keyword **`MINUS`**. Their logical behaviors are fully identical.

```
-- Syntax Blueprint (Oracle Variant)
SELECT column1, column2 FROM table_a
MINUS
SELECT column1, column2 FROM table_b;
```

Practical Project Assignment Example:

Isolating stagnant or newly onboarded employees who are not linked to active corporate pipelines:

```
SELECT ssn                    AS "SSN",
       fname || ' ' || lname   AS "Name"
FROM   employee
MINUS
SELECT e.ssn,
       e.fname || ' ' || e.lname
FROM   employee e
JOIN   works_on w ON e.ssn = w.essn
ORDER BY "SSN";
```

Execution Order & Operator Precedence

When processing chain structures containing multiple mixed set operations, databases process transactions from top-to-bottom. However, **`INTERSECT`** exhibits a higher natural precedence evaluation rank relative to `UNION` or `MINUS` / `EXCEPT`. To control executions cleanly and avoid logical faults, wrap sequential execution workflows within structured parentheses:

```
(SELECT job_id FROM employees
 UNION
 SELECT job_id FROM job_history)
MINUS
SELECT job_id FROM jobs
WHERE min_salary > 10000;
```

Quick-Reference Summary Matrix

Operation	Result Logic	Duplicate Handling	Performance Impact
UNION	Combines records from both queries vertically.	**Removes** all duplicates	Moderate/Slow (requires explicit sort)
UNION ALL	Appends sets together uniformly.	**Preserves** all duplicates	Fast (direct data streaming)
INTERSECT	Isolates overlapping rows common to both.	Removes duplicate returns	Moderate (requires comparison hash)
EXCEPT / MINUS	Subtracts secondary query matches from primary.	Removes duplicate returns	Moderate (requires verification pass)